TinyShell: Diagnosing Data Shortcuts and the Limits of Scaling in a 10M-Parameter Natural-Language-to-Shell Model

TinyShell Contributors
Code, data pipeline, checkpoints, and evaluation logs:
`github.com/.../tinyshell` (this repository)

October 2026

Abstract

We present TinyShell, an end-to-end study of building natural-language-to-shell-command models from scratch on a CPU-only budget, culminating in a 9.6M-parameter decoder-only transformer trained on 100M tokens derived from the tldr-pages corpus. Because we control the data generator, we hold out three generalization axes independently—slot values, request phrasings, and entire command pages—and pair every neural result with a BM25 retrieval baseline. Three findings emerge. First, a widely used training recipe (naive sequence packing) teaches a shortcut: the model learns $P(\text{command} \mid \text{request}, \text{previous example})$ rather than $P(\text{command} \mid \text{request})$, reaching 0.000 exact match despite healthy validation loss; blocking cross-example attention at evaluation time on the frozen checkpoint improves validation loss by 0.56 nats, directly evidencing the shortcut, and retraining with document-aligned packing (block-diagonal masks with within-example position reset) removes it. Second, a controlled subset experiment shows the residual failure of a 2.1M-parameter model is decomposable: command-family selection is label-space limited (restricting 3,651 first tokens to 111 raises family selection from 0.36 to 0.87), while argument composition is capacity limited. Third, scaling 4.6× in parameters and 10.9× in tokens converts seen-command performance from non-functional to competitive (exact match 0.46–0.56, token F1 0.60–0.68, paraphrase-robust) but leaves unseen-page exact match at 0.005 while *regressing* unseen-page validation loss (3.80→4.15): phrasing diversity hyper-fits known pages rather than teaching new tools. On unseen pages the generative model reaches parity or better with retrieval on token F1 for the first time (+0.001 to +0.029), an initial signature of composition over memorization. We release the full pipeline—data cards, model cards, checkpoints, evaluation logs, and a learning book—as a reproducible testbed for composition, retrieval augmentation, and evaluation methodology at tiny scale.

1 Introduction

Mapping an English request to a single shell command is a bounded, checkable, and practically useful instance of semantic parsing: outputs are short (median 26 characters), correctness is measurable without human judges, and the resource a model must possess—a vocabulary of tools and flags—is small enough to study exhaustively. Prior work established the task and its difficulty with web-scale corpora and seq2seq semantic parsers [1, 2]; modern code LLMs solve it broadly but opaquely at billions of parameters [3, 4]. What is missing is a *small, fully instrumented* testbed in which every design decision—corpus construction, tokenization, packing, objective, evaluation split design, and baselines—is visible, ablatable, and honest about failure. This paper describes TinyShell (Figure 1), such a testbed, and the measurement-driven story of its four training runs.

Our contributions:

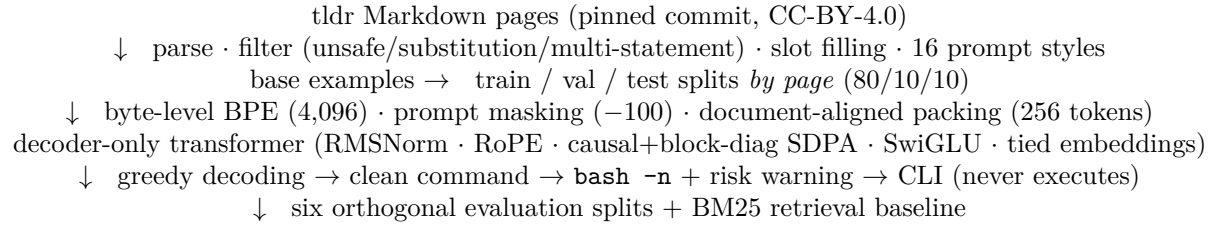


Figure 1: The TinyShell pipeline. Every stage is released code; every downstream number in this paper is reproducible from the pinned corpus commit and per-run metadata.

1. **A shortcut diagnosis with an eval-time causal test.** We show that naive sequence packing teaches cross-example copying, that validation loss conceals it, and that a *frozen-checkpoint mask ablation* (3.866 → 3.306 nats) plus a behavioral prefix probe identify the mechanism without retraining (Section 6).

2. **A label-space/capacity decomposition.** A focused-subject experiment (111 command families, 5× first-token loss weight) raises family selection from 0.36 to 0.87 with the same architecture, showing selection was label-space limited while argument composition remained capacity limited (Section 7).

3. **A scaling result with orthogonal generalization axes.** Scaling 2.1M → 9.6M parameters and 9M → 100M tokens at fixed page splits lifts seen-command exact match from ≤ 0.002 to 0.525–0.559, leaves unseen-page exact match at 0.005, and *worsens* unseen-page validation loss from 3.80 to 4.15—quantifying that phrasing-diverse data hyper-fits known pages (Section 8, Table 6, Table 7).

4. **A generative-vs-retrieval accounting.** A BM25 baseline reaches 0.84–0.88 exact on seen splits (evaluation requests are generated paraphrases of training data, making retrieval near-oracle there) and ∼0.008 on unseen pages; the 9.6M generative model matches or exceeds retrieval token F1 on all unseen splits (+0.001 to +0.029), the first evidence of compositional behavior at this scale (Table 7).

5. **A released, reproducible research platform**: data and model cards, pinned corpus commit, per-run environment and git metadata, checkpoints, raw per-example predictions, and a 16-chapter learning book documenting every experiment (Section 12).

2 Related Work

NL-to-shell. Lin et al. [1] built Tellina, an RNN-based semantic parser with slot filling for bash one-liners, and Lin et al. [2] released the NL2Bash corpus (∼9,305 expert-described commands, top-1 full-command accuracy 0.36). TinyShell differs in source (curated tldr-pages rather than scraped data), scale (from-scratch 2–10M parameters on CPU), and focus: generalization audit and baselines rather than leaderboard performance.

Instruction-tuned code models. Codex [3], Code Llama [4], Llama-family models [5, 6], and instruction tuning [7] solve shell questions broadly; we deliberately operate three orders of magnitude smaller to keep every mechanism measurable, in the spirit of TinyStories [8], TinyLlama [9], and nanoGPT [10].

Sequence packing. Packing variable-length samples into fixed-length rows is standard practice [11, 12]; Krell et al. [11] formalize packing as bin packing and show block-diagonal masking plus position adjustments preserve mathematical equivalence; document-boundary masking with position reset is also used in conditioned generation [13]. Our contribution is *diagnostic*: demonstrating that the contamination left in by naive packing is not a small accuracy

dent but a learned conditional that silently changes the task, measurable by evaluating a frozen checkpoint under both masks.

Scaling laws and data mixture. Scaling analyses [14, 15] concern tokens and parameters for fixed distributions; our result concerns *within-page variant scaling*, showing a regime where more tokens (all phrasings of known pages) improve interpolation and regress extrapolation. Retrieval-augmented generation [16] and interactive execution environments [17] frame our proposed remedies (Section 11).

3 Task and Data

3.1 Corpus

The sole knowledge source is tldr-pages [18], pinned at commit `95062b79` (`pages/common` + `pages/linux`; CC-BY-4.0): 6,803 pages / 30,308 raw examples yield 6,621 usable pages / 28,659 usable base examples after filtering. Each example is a one-line description plus a command template with `{{placeholder}}` slots. Filters reject multi-statement, substitution-heavy, history-expanded, interactive, and oversized commands; the run-3 revision additionally rejects cron-shaped first tokens, leading `!`, and process substitution (−139 examples). Slots are filled from hand-written pools (file, path, user, host, package, port, pattern, version) with per-variant fresh values and tldr alternative-syntax resolution (`{{[-a|--all]}}`).

3.2 Requests, styles, and splits

Each base example is rendered into natural-language requests by 16 templates ("styles"): 12 train styles (indices 0–7 plus 12–15), with styles 8–11 *held out* for validation and testing. Slot mentions inlined in the description are substituted inline; remaining slots are declared in an appended context sentence ("Assume the file is `/tmp/notes.txt`.").

Splits are by *page* (deterministic hash of page title, 80/10/10), so all examples of a command fall in exactly one split. Six evaluation splits of 1,500 requests each (v1; identical page split reused in run 3) orthogonalize three axes:

Split	Pages	Prompt style	Measures
`val`	seen	8 (held out)	checkpoint selection (loss)
`val_unseen`	unseen	seen	command generalization (loss)
`test_seen`	seen	9 (held out) + new slots	slot filling + style robustness
`test_paraphrase`	seen	10–11 (unseen)	phrasing robustness
`test_unseen`	unseen	seen	command generalization
`test_unseen_paraphrase`	unseen	10–11	both axes held out

Table 1: Evaluation splits. Holding out slots, styles, and pages independently is what allows the scaling analysis of Section 8 to attribute gains and failures precisely.

3.3 Versions

Dataset v1 (runs 1–2, core): 6 variants per base example → 135,860 training requests → 9.16M packed tokens per epoch. Dataset XL (run 3): 150 variants per example with 4 additional train styles → 1,303,083 requests → 99,992,950 packed training tokens in 468,212 document-aligned chunks (17,411,259 supervised label tokens; max example 229 tokens). Slot-free examples saturate at 12 distinct phrasings; slot-bearing examples scale near-linearly. The same page split is preserved, keeping cross-run comparisons directional. Risk labels (129,177 safe / 14,822 caution / 861 destructive) are regex heuristics used for analysis and CLI warnings, never as training targets; `bash -n` validity of a 2,000-command sample is 99.85%.

4 Model and Objective

TinyShell is a pre-norm decoder-only transformer [5, 19]: RMSNorm [20], RoPE [21], fused scaled-dot-product attention [12], SwiGLU MLPs [22], and tied input/output embeddings [23]. Byte-level BPE [24] with a 4,096 vocabulary (five reserved special tokens) and a fixed three-role chat template (system/user/assistant) are shared by all runs; prompt tokens are masked with -100 so the loss covers only the command and EOS. Positions are per-example (restart at 0 at every packed document), and attention is causal *and* block-diagonal over packed documents (Section 6).

	2.1M (runs 1–2, core)	9.6M (run 3)
d_{model} / layers / heads	128 / 6 / 4	256 / 10 / 8
Head dim	32	32
SwiGLU intermediate	512	768
Context / vocab	256 / 4,096	256 / 4,096
Dropout	0.05 (core 0.1)	0.05
Total parameters	2,098,816	**9,573,632**
Non-embedding	1,574,528	8,525,056
Embedding share	25.0%	11.0%

Table 2: Architectures. Head dimension is fixed at 32 across scales to keep the capacity comparison clean.

Objective. Token cross-entropy on supervised positions, averaged token-weighted. An optional *first-token weight* w multiplies the loss on the first command token of every packed example (the command-family decision); w=5.0 in the core diagnostic, w=1.0 elsewhere.

5 Training Setup

AdamW [25] with decoupled decay 0.1 on matrices only; β=(0.9, 0.95); lr 3e$-$4 cosine-annealed to 3e$-$5 after linear warmup (200–300 steps); gradient clipping at global norm 1.0; fp32 on 4 vCPU ARM64 (Neoverse-N1), 4 threads. Examples are packed complete-only into 256-token rows ($\sim$14% padding), memory-mapped as `uint16`/`int32`/`int16` arrays. Checkpoints store model, optimizer, scheduler, and progress (resumable); best-checkpoint selection uses `val` (held-out style 8 on seen pages). Seed 42 throughout; per-run snapshots of config, environment (Python 3.12.3, torch 2.14.1+cpu), git commit, and tokenizer are stored alongside logs.

6 The Packing Shortcut and Its Repair

Run 1 (naive packing). Concatenating all examples into a continuous stream and slicing into 256-token rows lets position t of example N attend to the *command of example* $N-1$. The model trained to plausible loss (best val 3.9749) but generated 0.000 exact match with 72% of predictions beginning with `sudo` (references: 6%), e.g. *"list all files including hidden ones"* $\rightarrow$ `sudo aa-cli --list`.

Diagnosis without retraining. Two probes on the frozen checkpoint: (i) *Prefix probe*—prepending an unrelated prior example to the prompt flips the prediction to copy it; (ii) *mask ablation*—evaluating the same packed validation set with a block-diagonal document mask lowers validation loss from 3.866 to 3.306 (-0.56 nats). If preceding examples were useful context, masking them would *hurt*; the gain proves a harmful learned dependence. Both probes are possible because document ids are a runtime input.

Run 2 (document-aligned packing). We then retrained with complete-example packing, per-token document ids, within-example positions, and the block-diagonal causal mask—the

train-time counterpart of the eval mask, and the standard remedy for cross-contamination [11]. Same architecture, data, and optimizer.

	Run 1 (naive)	Run 2 (doc-aligned)	Δ
Best val loss	3.9749	**2.9377**	−1.04
Final val loss	3.9771	**2.9214**	−1.06
Val loss, unseen pages	4.6025	**3.7993**	−0.80
Exact match (best split)	0.002	0.002	≈
Command match (best split)	0.089	0.089	≈
Token F1 (best split)	0.128	0.187	+0.06

Table 3: Run 1 vs. run 2: removing the shortcut lowers loss by > 1 nat and improves argument copying everywhere, but exact match remains ≈ 0: the remaining gap is capacity/label-space, not masking.

The `sudo` prior persists (42% of run-2 `test_seen` predictions vs. 6.8% of references), now traceable to data frequency rather than a copying shortcut.

7 Decomposing the Failure: Label Space vs. Capacity

We build a focused subset—pages of ∼100 curated common tools: 655 pages, 17,317 requests, **111 distinct first command tokens** (vs. 3,651 broad)—and train the identical 2.1M model for 20 epochs with dropout 0.1 and first-token weight 5.0 (2.2 h).

Split (core)	N	Exact	Command	Token F1	Valid bash
val	187	0.048	**0.866**	0.227	0.952
test_seen	179	0.028	**0.737**	0.155	0.983
test_paraphrase	211	0.057	**0.924**	0.260	0.981

Table 4: Core-subject diagnostic. Command-family selection rises from 0.36 (the broad run-2 model scored on the same examples) to 0.74–0.92; `sudo` over-prediction drops from 42% to 6.7%; exact match stays low (0.028–0.057) because argument composition remains the bottleneck (71% of `test_seen` errors are right-family-wrong-arguments).

Interpretation. Family selection is *label-space limited*; argument composition is *model/data limited*. This decomposition directed run 3's design budget toward capacity and data volume rather than architecture changes.

8 Scaling: 9.6M Parameters, 100M Tokens

Run 3 scales the proven design (4.6× parameters) over the XL corpus (10.9× tokens; 150 variants/example; stricter filters; four extra train styles; held-out eval styles unchanged; same page split), trained one pass in 27.7 h at ∼1,190 tokens/s: 3,657 steps, final train loss ≈ 0.13, best val loss **0.6565** (perplexity 1.93) at step 3,500.

8.1 Loss results, and the headline regression

8.2 Generation results

8.3 Error census

On `test_seen` (1,500): 788 exact (52.5%), 250 right-command-wrong-arguments (16.7%), 4 invalid bash (0.3%), 0 empty. The `sudo` prior persists at 14.7% of predictions vs. 5.5%

	Run 1	Run 2	Run 3
Best val loss (seen pages)	3.9749	2.9377	**0.6565**
Val loss, unseen pages	4.6025	3.7993	4.1451
Seen/unseen gap (nats)	0.63	0.87	**3.49**

Table 5: Validation loss. Run 3 *regresses* on unseen pages: 150 variants per page hyper-fit known command families. The seen/unseen loss gap widens from 0.87 to 3.49 nats.

Split	Exact	Command	Token F1	Valid bash	p50 lat.
val	0.460	0.618	0.604	0.997	290 ms
val_unseen	0.011	0.138	0.315	0.995	227 ms
test_seen	**0.525**	**0.692**	**0.644**	0.997	245 ms
test_paraphrase	**0.559**	**0.722**	**0.681**	0.997	242 ms
test_unseen	0.005	0.111	0.289	0.989	229 ms
test_unseen_paraphrase	0.005	0.098	0.292	0.991	242 ms

Table 6: Run 3 (`best` checkpoint, greedy, 1,500/split). Seen-command composition now works; paraphrase is the *strongest* split; unseen-page exact match is unchanged from run 2's ≈ 0.

of references (run 2: 42%), concentrated on obscure admin pages (`zeroclaw`, `pveum`, `koji`)—plausible-prior, wrong-context behavior. Seen-split residue is dominated by sibling-tool confusions (`phpcs`↔`phpcbf`, `7zr`↔`7za`, `bun`↔`npm`) and invented flags on right families.

9 Analysis

What scaling fixed. 4.6×/10.9× moved the broad model from "usually the wrong tool" ($\leq 2\%$ exact) to a competitive suggestion engine on seen commands (0.525–0.559 exact under unseen phrasings), confirming that interpolation (slot filling, paraphrase) is an interpolation-feasible skill: copying context entities into a learned template.

What scaling did not fix. Unseen-page exact match stayed at 0.005 and unseen-page loss *regressed* (3.80→4.15). Scaling along the *phrasing* axis sharpens the fit to known pages; it does not supply the missing knowledge (flag vocabularies of unseen tools). The theoretically expected behavior of a data-limited conditional LM—falling back on a command-token prior exactly where evidence is absent—is what the model exhibits (`sudo` + wrong tool on obscure pages).

Retrieval as the control. BM25's seen-split dominance (0.857 mean exact vs. 0.515) is an artifact of paraphrase-generated evaluation and must be reported to contextualize generative claims; its unseen-split failure ($\sim 0.8\%$) marks the boundary of pure memorization. The model's unseen-page F1 crossing above retrieval's is small but conceptually important: a 9.6M generative model composes *something* about pages it never saw, which a retriever structurally cannot.

Methodological findings. (i) Validation loss concealed a task-level bug until generation probes and a frozen-checkpoint mask ablation exposed it; (ii) orthogonal holdout axes made the scaling result interpretable; (iii) the label-space diagnostic redirected the scaling budget; (iv) every qualitative claim is backed by released per-example prediction logs.

10 Limitations

Evaluation requests are synthetic paraphrases of a curated corpus (tldr common/linux only), with slot values from small pools—real-user performance is likely lower, and slot-robustness may be partly pool memorization. Exact match grants no semantic equivalence (a different-but-valid command scores 0), `bash -n` checks syntax only, risk labels are heuristics, and nothing is

	BM25			Model (run 3)		
Split	Exact	Cmd	F1	Exact	Cmd	F1
val	0.851	0.932	0.947	0.460	0.618	0.604
val_unseen	0.009	0.135	0.286	0.011	**0.138**	**0.315**
test_seen	0.843	0.918	0.918	0.525	0.692	0.644
test_paraphrase	0.877	0.942	0.964	0.559	0.722	0.681
test_unseen	0.007	0.124	0.277	0.005	0.111	**0.289**
test_unseen_paraphrase	0.008	0.114	0.291	0.005	0.098	**0.292**

Table 7: Generative model vs. BM25 retrieval [26] on identical examples. Retrieval is near-oracle on seen splits by construction (eval requests are paraphrases of training requests) and near-useless on unseen pages (0.8% exact). Bold marks the better side per column on unseen splits: the generative model edges retrieval on token F1 on all three unseen splits (+0.001 to +0.029) and on command match for `val_unseen`.

ever executed. The model hallucinates flags, cannot verify local tool versions, and must not be given execution authority; the CLI never runs generated commands and displays pattern-based risk warnings. Observed safety-relevant behavior (privilege-escalation prior on obscure pages) is disclosed above and is motivation for the calibration work proposed next. Run 3 is not a controlled ablation of run 2 (corpus and filters change simultaneously); the page-split overlap makes cross-run comparisons directional, not exact. The core run's first-token weighting implementation tagged only the first active token per chunk (about one in three examples), so its reported effect is a lower bound.

11 Future Work

- **Retrieval-augmented generation** [16]: condition the prompt on the retrieved tldr page of the (unseen) tool, converting "unseen page" into recombination of retrieved evidence; our BM25 index and six-split harness are the ready-made evaluation.
- **Data-mix curriculum**: cap variants per page and spend tokens on more pages (or rare-first-token oversampling) at fixed 100M tokens, testing whether the unseen-page loss regression (Table 5) is an artifact of mixing.
- **Execution-grounded training**: sandboxed, reversible execution feedback [17] to attack the 16.7% right-family-wrong-arguments rate (rejection-sampling SFT or RL on executable-verifiable requests).
- **Calibration and abstention**: per-request sequence log-prob as an uncertainty signal to route low-confidence cases to retrieval or an explicit "I don't know," directly targeting the `sudo` prior on obscure pages.
- **Semantic-equivalent evaluation**: executable-equivalence rubrics beyond exact match, addressing the strictness that makes even near-miss sibling tools score 0.
- **Scaling the coverage axis**: parameterize unseen-page loss by *distinct pages* rather than tokens—a "coverage law" for tiny models; and byte/char-level tokenization ablations for rare-tool spelling.
- **Mechanistic probes**: head-level attribution and activation patching (cheap at this scale) to localize where argument composition fails.

12 Released Platform

The repository contains: the full data pipeline (pinned corpus commit, manifests, data card), tokenizer and packing code with tested invariants, the 272-line model, training/evaluation/baseline code, per-run metadata (config, environment, git commit, tokenizer snapshot), checkpoints (model-only `best-model.pt` files for all four runs), raw per-example predictions and failure logs for every evaluation, a packaged self-contained model, CLI with retrieval mode and risk warnings, a zero-dependency training dashboard, and a 16-chapter learning book that includes all theory, implementation walkthroughs, run narratives, and reproduction commands. We release everything under MIT (code) with tldr data under CC-BY-4.0 attribution.

13 Conclusion

TinyShell demonstrates that a 10M-parameter model trained on one CPU-day can become a competitive suggestion engine *within* its training distribution—and that the same disciplined evaluation that proves it also proves where it ends: page-level generalization. The packed-shortcut diagnosis, the label-space/capacity decomposition, and the scaling-with-regression result are each individually small; together they form a template for honest, mechanism-level study of small language models, and a platform on which the next questions—retrieval augmentation, execution grounding, calibration, and coverage scaling—are already instrumented.

Reproducibility

All numbers in this paper are generated by the repository's code and stored in `artifacts/` (evaluation `metrics.json`/`report.md`, training `metrics.jsonl`/`train.log`, per-run `env.json`/`git_commit.txt`). Chapter 15 of the learning book lists the exact commands, resolved configurations, and artifact locations.

References

[1] Xi Victoria Lin, Chenglong Wang, Deric Pang, Kevin Vu, Luke Zettlemoyer, and Michael D. Ernst. Program synthesis from natural language using recurrent neural networks. Technical Report UW-CSE-17-03-01, University of Washington, Department of Computer Science and Engineering, 2017.

[2] Xi Victoria Lin, Chenglong Wang, Luke Zettlemoyer, and Michael D. Ernst. NL2Bash: A corpus and semantic parser for natural language interface to the Linux operating system. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC)*, 2018. arXiv:1802.08979.

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[4] Baptiste Rozière, Jonas Gehring, Fabrice Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tom Raffel, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

[5] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[6] Aaron Grattafiori et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[7] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022. arXiv:2203.02155.

[8] Ronen Eldan and Yuanzhi Li. TinyStories: Small language models that can tell stories. *arXiv preprint arXiv:2305.07759*, 2023.

[9] Peiyuan Zhang, Guangtao Zeng, Tiandi Li, Howard Wang, Lu Zhang, and Charles Zee. TinyLlama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024.

[10] Andrej Karpathy. nanoGPT: The simplest, fastest repository for training/finetuning medium-sized GPTs. https://github.com/karpathy/nanoGPT, 2023.

[11] Mario Michael Krell, Matej Kosec, Sergio P. Perez, and Andrew Fitzgibbon. Efficient sequence packing without cross-contamination: Accelerating large language models without impacting performance. *arXiv preprint arXiv:2107.02027*, 2021.

[12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness, 2022. arXiv:2205.14135.

[13] Jade Copet, Felix Kreuk, Itai Gat, Tal Remez, David Kant, Gabriel Synnaeve, Yossi Adi, and Alexandre Défossez. Simple and controllable music generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. arXiv:2306.05284.

[14] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[15] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hinton, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Jack W. Rae, Oriol Vinyals, Laurent Sifre, Seb Tighe, Sander Merity, Simon Hadfield, Charles Sumner, Sona Ortis-Baban, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022. arXiv:2203.15556.

[16] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. arXiv:2005.11401.

[17] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. InterCode: Standardizing and benchmarking interactive coding with execution feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. arXiv:2306.14898.

[18] tldr-pages contributors. tldr-pages: Simplified and community-driven man pages. https://github.com/tldr-pages/tldr, 2024. Licensed CC-BY-4.0; snapshot commit 95062b79e30fc3539246399b1dbfcfabbccc79cf.

[19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017. arXiv:1706.03762.

[20] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. arXiv:1910.07467.

[21] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[22] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[23] Ofir Press and Lior Wolf. Using the output embedding to improve language models. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pages 157–163, 2017. arXiv:1608.05859.

[24] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725, 2016. arXiv:1508.07909.

[25] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1711.05101.

[26] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.